

	Departamento: Ciencias Computacionales
	Carrera: ING.Electronica y automatizacion Asignatura: Fundamentos de la programacion NRC: 28561
	Apellidos y nombres del estudiante: Comina Francisco

ANTECEDENTES

Las cadenas de texto son una de las estructuras de datos más utilizadas en programación. Su manipulación permite realizar tareas como contar caracteres, verificar palabras, unir textos y buscar información dentro de una cadena, facilitando el procesamiento de datos ingresados por el usuario.

OBJETIVO

Desarrollar algoritmos que permitan realizar operaciones básicas con cadenas de texto, como contar letras y espacios, identificar palíndromos, unir cadenas y buscar palabras dentro de un texto, aplicando estructuras de control y funciones de manejo de cadenas.

DESARROLLO

Mini-práctica (para clase/laboratorio)

1. Leer una frase y contar cuántas letras y cuántos espacios tiene.

```
#include <stdio.h>
#include <string.h>

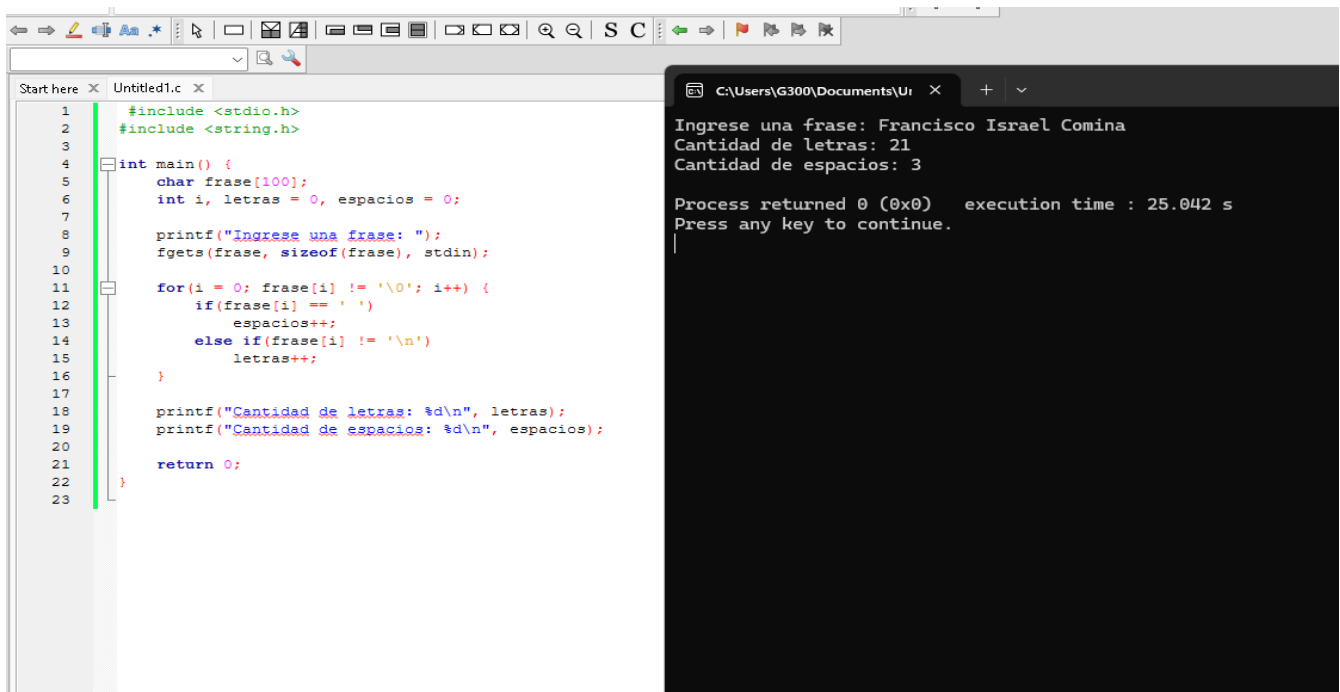
int main() {
    char frase[100];
    int i, letras = 0, espacios = 0;

    printf("Ingrese una frase: ");
    fgets(frase, sizeof(frase), stdin);

    for(i = 0; frase[i] != '\0'; i++) {
        if(frase[i] == ' ')
            espacios++;
        else if(frase[i] != '\n')
            letras++;
    }

    printf("Cantidad de letras: %d\n", letras);
    printf("Cantidad de espacios: %d\n", espacios);

    return 0;
}
```



2. Validar si una cadena es un palíndromo (ignorando espacios).

```
#include <stdio.h>
#include <string.h>

int main() {
    char cadena[100];
    char limpia[100];
    int i, j = 0;
    int esPalindromo = 1;

    printf("Ingrese una cadena: ");
    fgets(cadena, sizeof(cadena), stdin);

    // Eliminar espacios y salto de línea
    for(i = 0; cadena[i] != '\0'; i++) {
        if(cadena[i] != ' ' && cadena[i] != '\n') {
            limpia[j] = cadena[i];
            j++;
        }
    }
    limpia[j] = '\0';

    // Verificar si es palíndromo
    for(i = 0; i < strlen(limpia) / 2; i++) {
        if(limpia[i] != limpia[strlen(limpia) - 1 - i]) {
            esPalindromo = 0;
            break;
        }
    }

    if(esPalindromo)
        printf("La cadena es un palindromo.\n");
    else
        printf("La cadena no es un palindromo.\n");

    return 0;
}
```

}

The image shows a C program in a code editor on the left and its execution in a terminal window on the right. The code in the editor is a program to check if a string is a palindrome. It includes `<stdio.h>` and `<string.h>`. In the `main` function, it declares `cadena` and `limpia` as `char` arrays of size 100, initializes `i` and `j` to 0, and sets `esPalindromo` to 1. It prompts the user to enter a string using `printf` and `fgets`. Then, it removes spaces and newlines from the string by copying non-space characters from `cadena` to `limpia`. Afterward, it checks if the string is a palindrome by comparing characters from both ends towards the center. If it is a palindrome, it prints "La cadena es un palindromo.\n"; otherwise, it prints "La cadena no es un palindromo.\n". Finally, it returns 0.

```
#include <stdio.h>
#include <string.h>

int main() {
    char cadena[100];
    char limpia[100];
    int i, j = 0;
    int esPalindromo = 1;

    printf("Ingrese una cadena: ");
    fgets(cadena, sizeof(cadena), stdin);

    // Eliminar espacios y salto de línea
    for(i = 0; cadena[i] != '\0'; i++) {
        if(cadena[i] != ' ' && cadena[i] != '\n') {
            limpia[j] = cadena[i];
            j++;
        }
    }
    limpia[j] = '\0';

    // Verificar si es palindromo
    for(i = 0; i < strlen(limpia) / 2; i++) {
        if(limpia[i] != limpia[strlen(limpia) - 1 - i]) {
            esPalindromo = 0;
            break;
        }
    }

    if(esPalindromo)
        printf("La cadena es un palindromo.\n");
    else
        printf("La cadena no es un palindromo.\n");

    return 0;
}
```

The terminal window on the right shows the program's output. It prompts "Ingrese una cadena:" and the user enters "anita lava la tina". The program then outputs "La cadena es un palindromo." and "Process returned 0 (0x0) execution time : 6.538 s Press any key to continue."

3. Unir nombre y apellido en una sola cadena con un espacio entre ambos.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
    char nombre[40], apellido[40], completo[80];
```

```
    printf("Ingrese el nombre: ");
    scanf("%s", nombre);
```

```
    printf("Ingrese el apellido: ");
    scanf("%s", apellido);
```

```
    strcpy(completo, nombre);
    strcat(completo, " ");
    strcat(completo, apellido);
```

```
    printf("Nombre completo: %s\n", completo);
```

```
    return 0;
}
```

The image shows a C program in a code editor and its execution in a terminal. The code in the editor is as follows:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char nombre[40], apellido[40], completo[80];
6
7     printf("Ingrese el nombre: ");
8     scanf("%s", nombre);
9
10    printf("Ingrese el apellido: ");
11    scanf("%s", apellido);
12
13    strcpy(completo, nombre);
14    strcat(completo, " ");
15    strcat(completo, apellido);
16
17    printf("Nombre completo: %s\n", completo);
18
19    return 0;
20 }
21
```

The terminal output shows the program's execution with the following text:

```
C:\Users\G300\Documents\Ur
Ingrese el nombre: Francisco
Ingrese el apellido: Fonseca
Nombre completo: Francisco Fonseca

Process returned 0 (0x0)   execution time : 9.760 s
Press any key to continue.
```

4. Buscar una palabra dentro de un texto e indicar la posición donde aparece por primera vez.

```
#include <stdio.h>
#include <string.h>

int main() {
    char texto[200], palabra[50];
    char *posicion;

    printf("Ingrese un texto: ");
    fgets(texto, sizeof(texto), stdin);

    printf("Ingrese la palabra a buscar: ");
    fgets(palabra, sizeof(palabra), stdin);

    // Eliminar el salto de línea de la palabra
    palabra[strcspn(palabra, "\n")] = '\0';

    posicion = strstr(texto, palabra);

    if (posicion != NULL) {
        printf("La palabra aparece por primera vez en la posicion: %d\n",
            (int)(posicion - texto + 1));
    } else {
        printf("La palabra no fue encontrada.\n");
    }

    return 0;
}
```

The image shows a C program in Visual Studio and its execution in the terminal. The program is a simple string search utility. It prompts the user to enter a text and a word to search for. It then finds the first occurrence of the word in the text and prints its position (1-based index). If the word is not found, it prints a message indicating that.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char texto[200], palabra[50];
6     char *posicion;
7
8     printf("Ingrese un texto: ");
9     fgets(texto, sizeof(texto), stdin);
10
11     printf("Ingrese la palabra a buscar: ");
12     fgets(palabra, sizeof(palabra), stdin);
13
14     // Eliminar el salto de línea de la palabra
15     palabra[strcspn(palabra, "\n")] = '\0';
16
17     posicion = strstr(texto, palabra);
18
19     if (posicion != NULL) {
20         printf("La palabra aparece por primera vez en la posición: %d\n",
21             (int)(posicion - texto + 1));
22     } else {
23         printf("La palabra no fue encontrada.\n");
24     }
25
26     return 0;
27 }
28
```

Terminal Output:

```
C:\Users\G300\Documents\U...
Ingrese un texto: lola come trigo en un trigel
Ingrese la palabra a buscar: lola
La palabra aparece por primera vez en la posición: 1

Process returned 0 (0x0)   execution time : 47.441 s
Press any key to continue.
```

